



UNIVERSIDAD NACIONAL DEL ALTIPLANO DE PUNO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA



Curso: Estructura de Datos (2026)

Docente: Prof. Fred Torres Cruz

Estudiante: Henry Antonio Mamani Gutierrez

Correo: henrymamani@est.unap.edu.pe

Repositorio GitHub Oficial: <https://github.com/henrymamani-alt/henry-ejercicios>

TRABAJO ENCARGADO - N° 001: ESTRUCTURAS BÁSICAS

Presentación: A continuación se detalla el desarrollo formal de los quince (15) ejercicios del Capítulo I (Estructuras Básicas). Para cada ejercicio se incluye el enunciado oficial, el análisis computacional del algoritmo, el código fuente completo en C/C++, el enlace individual de acceso al repositorio en GitHub y la captura gráfica de evidencia de ejecución exitosa en terminal.

Ejercicio 1: Cálculo de Costo de Procesamiento por Analista

Enunciado: Desarrolle un programa en C que registre las horas dedicadas por un analista al procesamiento de un conjunto de datos y el costo por hora. Calcule el costo total del procesamiento y muestre un resumen con las horas trabajadas, tarifa aplicada y costo final.

Análisis y Fundamentación Algorítmica:

Se solicitan dos variables numéricas en coma flotante (float): `horas_trabajadas` y `costo_por_hora`. La tarifa final es el producto directo de ambos valores: $\text{Costo Total} = \text{Horas Trabajadas} \times \text{Tarifa por Hora}$. Se valida que ambas entradas sean valores no negativos.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio1.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    float horas_trabajadas;
    float costo_por_hora;
    float costo_total;

    printf("=====\n");
    printf("  CALCULO DE COSTOS DE PROCESAMIENTO DE DATOS    \n");
    printf("=====\n");

    printf("Ingrese las horas dedicadas por el analista: ");
    if (scanf("%f", &horas_trabajadas) != 1 || horas_trabajadas < 0) {
        printf("Error: Entrada invalida para horas trabajadas.\n");
        return 1;
    }

    printf("Ingrese la tarifa (costo por hora en S/.): ");
    if (scanf("%f", &costo_por_hora) != 1 || costo_por_hora < 0) {
        printf("Error: Entrada invalida para costo por hora.\n");
        return 1;
    }

    costo_total = horas_trabajadas * costo_por_hora;

    printf("\n----- RESUMEN DE PROCESAMIENTO ----- \n");
    printf(" Horas trabajadas : %.2f hrs\n", horas_trabajadas);
    printf(" Tarifa aplicada  : S/. %.2f por hora\n", costo_por_hora);
    printf(" Costo total final: S/. %.2f\n", costo_total);
    printf("----- \n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
=====
      CALCULO DE COSTOS DE PROCESAMIENTO DE DATOS
=====
Ingrese las horas dedicadas por el analista: Ingrese la tarifa (costo por hora en S/.):
----- RESUMEN DE PROCESAMIENTO -----
Horas trabajadas : 42.50 hrs
Tarifa aplicada  : S/. 35.00 por hora
Costo total final: S/. 1487.50
-----
```

Ejercicio 2: Registro de Observación de Datos y Tipos Apropriados

Enunciado: Desarrolle un programa en C que registre información básica de una observación de un conjunto de datos: identificador, edad, valor promedio de una variable, categoría representada por una letra y estado de validez del registro. Utilice tipos de datos apropiados y muestre la información almacenada.

Análisis y Fundamentación Algorítmica:

Se seleccionan tipos de datos exactos: `int` para el identificador numérico y edad; `double` para el valor promedio continuo de 64 bits; `char` para la categoría cualitativa; y `bool` (de `<stdbool.h>`) para el estado lógico binario de validez.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio2.c>

Código Fuente (C):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int id;                // Identificador unico (entero)
    int edad;              // Edad de la observacion (entero)
    double valor_promedio; // Valor promedio de una variable continua (flotante doble precision)
    char categoria;        // Categoria representada por una letra (caracter)
    bool es_valido;        // Estado de validez del registro (booleano)
    int entrada_validez;

    printf("=====\n");
    printf("      REGISTRO DE OBSERVACION DE DATOS      \n");
    printf("=====\n");

    printf("Ingrese ID del registro (ej. 101): ");
    scanf("%d", &id);

    printf("Ingrese la edad del individuo: ");
    scanf("%d", &edad);

    printf("Ingrese el valor promedio de la variable (ej. 84.75): ");
    scanf("%lf", &valor_promedio);

    printf("Ingrese la categoria (letra A, B, C, etc.): ");
    scanf(" %c", &categoria);

    printf("Estado de validez (1 = Valido, 0 = Invalido): ");
    scanf("%d", &entrada_validez);
    es_valido = (entrada_validez != 0);

    printf("\n===== RESUMEN DE OBSERVACION =====\n");
    printf(" [1] Identificador (int)      : %d\n", id);
    printf(" [2] Edad (int)                : %d años\n", edad);
    printf(" [3] Valor Promedio (double)   : %.4f\n", valor_promedio);
    printf(" [4] Categoria (char)          : %c\n", categoria);
    printf(" [5] Estado de Validez (bool)  : %s\n", es_valido ? "VALIDO (true)" : "INVALIDO (false)");
    printf("=====\n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
Terminal Execution - Ejercicio 2 (C)

=====
REGISTRO DE OBSERVACION DE DATOS
=====
Ingrese ID del registro (ej. 1001): Ingrese la edad del individuo: Ingrese el valor promedio de la variable (ej. 84.75): Ingrese la categoria (letra A, B, C, etc.): Estado de validez (1 = Valido, 0 = Invalido):
===== RESUMEN DE OBSERVACION =====
[1] Identificador (int)      : 1001
[2] Edad (int)              : 21 años
[3] Valor Promedio (double)  : 17.8500
[4] Categoria (char)        : A
[5] Estado de Validez (bool) : VALIDO (true)
=====
```

Ejercicio 3: Conversión Explícita (Casting) y Pérdida Decimal

Enunciado: Desarrolle un programa en C que solicite una medición decimal obtenida por un sensor, por ejemplo 18.9, la convierta explícitamente a un valor entero y muestre ambos resultados. Indique mediante la salida del programa cuánto valor decimal se pierde durante la conversión.

Análisis y Fundamentación Algorítmica:

Se demuestra el operador de moldeo o casting `(int)medicion_decimal`, el cual trunca la parte fraccionaria. La pérdida resultante se cuantifica evaluando la diferencia: $\Delta = \text{Medición Decimal} - (\text{double})\text{Medición Entera}$.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio3.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    double medicion_decimal;
    int medicion_entera;
    double valor_perdido;

    printf("=====\n");
    printf("    CONVERSION EXPLICITA (CASTING) DE SENSOR    \n");
    printf("=====\n");

    printf("Ingrese la medicion decimal del sensor (ej. 18.9): ");
    if (scanf("%lf", &medicion_decimal) != 1) {
        printf("Error: Entrada invalida.\n");
        return 1;
    }

    // Conversion explicita (casting) a entero
    medicion_entera = (int)medicion_decimal;

    // Calculo del valor decimal que se pierde
    valor_perdido = medicion_decimal - (double)medicion_entera;

    printf("\n----- RESULTADOS DE CONVERSION ----- \n");
    printf(" Valor original (decimal) : %.4f\n", medicion_decimal);
    printf(" Valor convertido (entero): %d\n", medicion_entera);
    printf(" Valor decimal perdido    : %.4f\n", valor_perdido);
    printf(" Porcentaje de perdida    : %.2f%%\n", (valor_perdido / medicion_decimal) * 100.0);
    printf("----- \n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

Terminal Execution - Ejercicio 3 (C)

```
=====
      CONVERSION EXPLICITA (CASTING) DE SENSOR
=====
Ingrese la medicion decimal del sensor (ej. 18.9):
----- RESULTADOS DE CONVERSION -----
Valor original (decimal) : 18.9000
Valor convertido (entero): 18
Valor decimal perdido    : 0.9000
Porcentaje de perdida    : 4.76%
-----
```

Ejercicio 4: Operaciones Aritméticas entre Registros de Algoritmos

Enunciado: Desarrolle un programa en C que solicite dos valores enteros correspondientes al número de registros procesados por dos algoritmos. Calcule suma, diferencia, producto, división entera y residuo, y muestre los resultados de manera organizada.

Análisis y Fundamentación Algorítmica:

Se procesan las operaciones aritméticas básicas de C (+, -, *, /, %). Se incluye protección condicional ante la eventualidad de divisor igual a cero para evitar excepciones en tiempo de ejecución.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio4.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    int reg_algoritmo1;
    int reg_algoritmo2;

    printf("=====\n");
    printf("  COMPARATIVA Y ARITMETICA DE REGISTROS (ALGORITMOS)\n");
    printf("=====\n");

    printf("Ingrese registros procesados por Algoritmo 1: ");
    scanf("%d", &reg_algoritmo1);

    printf("Ingrese registros procesados por Algoritmo 2: ");
    scanf("%d", &reg_algoritmo2);

    int suma = reg_algoritmo1 + reg_algoritmo2;
    int diferencia = reg_algoritmo1 - reg_algoritmo2;
    long long producto = (long long)reg_algoritmo1 * reg_algoritmo2;

    printf("\n+-----+-----+\n");
    printf("| Operacion          | Resultado                |\n");
    printf("+-----+-----+\n");
    printf("| Suma (+)           | %-27d |\n", suma);
    printf("| Diferencia (-)     | %-27d |\n", diferencia);
    printf("| Producto (*)       | %-27lld |\n", producto);

    if (reg_algoritmo2 != 0) {
        int division_entera = reg_algoritmo1 / reg_algoritmo2;
        int residuo = reg_algoritmo1 % reg_algoritmo2;
        printf("| Division Entera (/)| %-27d |\n", division_entera);
        printf("| Residuo / Modulo(%)| %-27d |\n", residuo);
    } else {
        printf("| Division Entera (/)| Indefinida (division por 0) |\n");
        printf("| Residuo / Modulo(%)| Indefinido (modulo por 0)  |\n");
    }
    printf("+-----+-----+\n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
=====
  COMPARATIVA Y ARITMETICA DE REGISTROS (ALGORITMOS)
=====
Ingrese registros procesados por Algoritmo 1: Ingrese registros procesados por Algoritmo 2:
+-----+-----+
| Operacion      | Resultado      |
+-----+-----+
| Suma (+)       | 1660           |
| Diferencia (-) | 1420           |
| Producto (*)   | 184800         |
| Division Entera (/) | 12           |
| Residuo / Modulo(%) | 100          |
+-----+-----+
```

Ejercicio 5: Distribución de Registros en Nodos de Cómputo

Enunciado: Un conjunto de datos contiene un determinado número de registros que deben distribuirse entre varios nodos de procesamiento. Desarrolle un programa en C que solicite el número total de registros y el número de nodos, determine cuántos registros procesa cada nodo y cuántos quedan sin distribuir uniformemente.

Análisis y Fundamentación Algorítmica:

Se aplica la división euclidiana: $total = nodos \times q + r$. El cociente $total / num_nodos$ corresponde a la carga uniforme asignada a cada nodo y el módulo $total \% num_nodos$ a los registros sobrantes sin distribuir.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio5.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    int total_registros;
    int num_nodos;

    printf("=====\n");
    printf("    DISTRIBUCION DE CARGA ENTRE NODOS DE COMPUTO    \n");
    printf("=====\n");

    printf("Ingrese el numero total de registros del dataset: ");
    scanf("%d", &total_registros);

    printf("Ingrese el numero de nodos de procesamiento: ");
    scanf("%d", &num_nodos);

    if (num_nodos <= 0 || total_registros < 0) {
        printf("Error: El numero de nodos debe ser mayor a 0 y los registros no negativos.\n");
        return 1;
    }

    int registros_por_nodo = total_registros / num_nodos;
    int registros_restantes = total_registros % num_nodos;

    printf("\n----- REPARTO DE CARGA DE TRABAJO ----- \n");
    printf(" Total de registros recibidos      : %d\n", total_registros);
    printf(" Cantidad de nodos disponibles      : %d\n", num_nodos);
    printf(" Registros procesados por cada nodo: %d\n", registros_por_nodo);
    printf(" Registros sin distribuir uniforme : %d\n", registros_restantes);
    printf(" Verificacion (nodos*carga + resto): %d * %d + %d = %d\n",
        num_nodos, registros_por_nodo, registros_restantes,
        (num_nodos * registros_por_nodo + registros_restantes));
    printf("----- \n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
=====
DISTRIBUCION DE CARGA ENTRE NODOS DE COMPUTO
=====
Ingrese el numero total de registros del dataset: Ingrese el numero de nodos de procesamiento:
----- REPARTO DE CARGA DE TRABAJO -----
Total de registros recibidos      : 1025
Cantidad de nodos disponibles    : 8
Registros procesados por cada nodo: 128
Registros sin distribuir uniforme : 1
Verificacion (nodos*carga + resto): 8 * 128 + 1 = 1025
-----
```

Ejercicio 6: Modificación Secuencial de Variable Acumulada

Enunciado: Durante un proceso iterativo, una variable representa el número acumulado de registros válidos. Desarrolle un programa en C que inicialice la variable en 4, incremente su valor en 3 y posteriormente duplique el resultado. Muestre el estado de la variable después de cada operación.

Análisis y Fundamentación Algorítmica:

Se evalúa la traza de ejecución de una variable acumuladora y los operadores de asignación compuesta: 1) Inicialización en 4; 2) Incremento en 3 ($+= 3 \rightarrow 7$); 3) Duplicación ($*= 2 \rightarrow 14$).

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio6.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    int registros_acumulados;

    printf("=====\n");
    printf("  SEGUIMIENTO DE ESTADO DE VARIABLE ACUMULADA      \n");
    printf("=====\n");

    // 1. Inicializacion en 4
    registros_acumulados = 4;
    printf("[Paso 1] Inicializacion:\n");
    printf("      registros_acumulados = %d\n\n", registros_acumulados);

    // 2. Incremento en 3
    registros_acumulados += 3;
    printf("[Paso 2] Incremento en 3 (operador += 3):\n");
    printf("      registros_acumulados = %d\n\n", registros_acumulados);

    // 3. Duplicacion del resultado
    registros_acumulados *= 2;
    printf("[Paso 3] Duplicar resultado (operador *= 2):\n");
    printf("      registros_acumulados = %d\n\n", registros_acumulados);

    printf("=====\n");
    printf(" Estado final del acumulador de registros: %d\n", registros_acumulados);
    printf("=====\n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

Terminal Execution - Ejercicio 6 (C)

```
=====
SEGUIMIENTO DE ESTADO DE VARIABLE ACUMULADA
=====
[Paso 1] Inicializacion:
        registros_acumulados = 4

[Paso 2] Incremento en 3 (operador += 3):
        registros_acumulados = 7

[Paso 3] Duplicar resultado (operador *= 2):
        registros_acumulados = 14

=====
Estado final del acumulador de registros: 14
=====
```

Ejercicio 7: Control de Calidad y Condiciones Mínimas de Dataset

Enunciado: Un conjunto de datos será aceptado para análisis únicamente si contiene al menos 100 observaciones válidas y presenta un porcentaje de datos completos igual o superior al 70 %. Desarrolle un programa en C que solicite ambos valores y determine si el conjunto de datos cumple las condiciones mínimas para ser analizado.

Análisis y Fundamentación Algorítmica:

Se evalúa la conjunción lógica: $(\text{observaciones} \geq 100) \ \&\& \ (\text{porcentaje} \geq 70.0)$. El sistema dictamina la admisión del conjunto o desglosa las causas específicas de rechazo.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio7.c>

Código Fuente (C):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int observaciones_validas;
    double porcentaje_completos;

    printf("=====\n");
    printf("      CONTROL DE CALIDAD DE CONJUNTO DE DATOS      \n");
    printf("=====\n");

    printf("Ingrese la cantidad de observaciones validas: ");
    scanf("%d", &observaciones_validas);

    printf("Ingrese el porcentaje de datos completos (0 - 100 %): ");
    scanf("%lf", &porcentaje_completos);

    // Criterios minimos: obs >= 100 y pct >= 70.0%
    bool cumple_obs = (observaciones_validas >= 100);
    bool cumple_pct = (porcentaje_completos >= 70.0);
    bool aceptado = cumple_obs && cumple_pct;

    printf("\n----- EVALUACION DE CRITERIOS ----- \n");
    printf(" 1. Observaciones validas (>= 100): %d [%s]\n",
        observaciones_validas, cumple_obs ? "CUMPLE" : "NO CUMPLE");
    printf(" 2. Datos completos (>= 70.00%)    : %.2f%% [%s]\n",
        porcentaje_completos, cumple_pct ? "CUMPLE" : "NO CUMPLE");
    printf("----- \n");

    if (aceptado) {
        printf(" >> RESULTADO: CONJUNTO DE DATOS ACEPTADO PARA ANALISIS.\n");
    } else {
        printf(" >> RESULTADO: CONJUNTO DE DATOS RECHAZADO.\n");
        printf("      Motivo(s): ");
        if (!cumple_obs) printf("[Faltan observaciones minimas] ");
        if (!cumple_pct) printf("[Porcentaje de completitud insuficiente] ");
        printf("\n");
    }
    printf("----- \n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
Terminal Execution - Ejercicio 7 (C)

=====
CONTROL DE CALIDAD DE CONJUNTO DE DATOS
=====
Ingrese la cantidad de observaciones validas: Ingrese el porcentaje de datos completos (0 - 100 %):
----- EVALUACION DE CRITERIOS -----
1. Observaciones validas (>= 100): 145 [CUMPLE]
2. Datos completos (>= 70.00%) : 88.50% [CUMPLE]
-----
>> RESULTADO: CONJUNTO DE DATOS ACEPTADO PARA ANALISIS.
-----
```

Ejercicio 8: Incorporación de Observaciones con Operadores Lógicos

Enunciado: Para incorporar una observación a un análisis, se requiere que el individuo tenga una edad mayor o igual que 18 años y que el registro haya sido validado. Desarrolle un programa en C que solicite ambos datos y utilice operadores lógicos para determinar si la observación puede incorporarse al conjunto de datos.

Análisis y Fundamentación Algorítmica:

Se implementan operadores relacionales y el operador lógico AND (&&): (edad ≥ 18) && (validado == 1), aplicando la regla de admisión de observaciones al análisis estadístico.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio8.c>

Código Fuente (C):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int edad;
    int estado_validacion; // 1 = Validado, 0 = No validado

    printf("=====\n");
    printf("    FILTRO DE ADMISION DE OBSERVACIONES AL ESTUDIO    \n");
    printf("=====\n");

    printf("Ingrese la edad del individuo: ");
    scanf("%d", &edad);

    printf("Ha sido validado el registro? (1 = Si / 0 = No): ");
    scanf("%d", &estado_validacion);

    bool es_mayor_edad = (edad >= 18);
    bool esta_validado = (estado_validacion == 1);

    // Operador logico AND (&&)
    bool puede_incorporarse = es_mayor_edad && esta_validado;

    printf("\n----- VERIFICACION DE CONDICIONES ----- \n");
    printf(" Condicion 1 (Edad >= 18)    : %d anios -> %s\n", edad, es_mayor_edad ? "VERDADERO" : "FALSO");
    printf(" Condicion 2 (Validado == 1): %d -> %s\n", estado_validacion, esta_validado ? "VERDADERO" : "FALSO");
    printf(" Expresion logica            : (edad >= 18) && (validado == 1)\n");
    printf(" Evaluacion                  : %s\n", puede_incorporarse ? "TRUE" : "FALSE");
    printf("----- \n");

    if (puede_incorporarse) {
        printf(" >> DICTAMEN: La observacion PUEDE incorporarse al conjunto de datos.\n");
    } else {
        printf(" >> DICTAMEN: La observacion NO PUEDE incorporarse al analisis.\n");
    }
    printf("=====\n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
=====
      FILTRO DE ADMISION DE OBSERVACIONES AL ESTUDIO
=====
Ingrese la edad del individuo: Ha sido validado el registro? (1 = Si / 0 = No):
----- VERIFICACION DE CONDICIONES -----
Condicion 1 (Edad >= 18)   : 20 años -> VERDADERO
Condicion 2 (Validado == 1): 1 -> VERDADERO
Expresion logica          : (edad >= 18) && (validado == 1)
Evaluacion                : TRUE
-----
>> DICTAMEN: La observacion PUEDE incorporarse al conjunto de datos.
=====
```

Ejercicio 9: Clasificación Climática de Temperatura por Sensor

Enunciado: Un sensor registra temperaturas ambientales utilizadas posteriormente en un análisis climático. Desarrolle un programa en C que solicite una temperatura y la clasifique como Congelación si es menor que 0 °C, Frío si está entre 0 y 20 °C, y Templado si supera los 20 °C.

Análisis y Fundamentación Algorítmica:

Se implementa una estructura condicional anidada if-else if-else para clasificar el dato en los tres rangos termoclimáticos: Congelación (< 0 °C), Frío (0 – 20 °C) o Templado (> 20 °C).

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio9.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    float temperatura;

    printf("=====\n");
    printf("      CLASIFICACION DE TEMPERATURAS CLIMATICAS      \n");
    printf("=====\n");

    printf("Ingrese la temperatura registrada por el sensor (en C): ");
    if (scanf("%f", &temperatura) != 1) {
        printf("Error: Registro de temperatura invalido.\n");
        return 1;
    }

    printf("\n----- RESULTADO DE EVALUACION ----- \n");
    printf(" Temperatura sensada: %.2f C\n", temperatura);
    printf(" Clasificacion      : ");

    if (temperatura < 0.0f) {
        printf("CONGELACION (< 0 C)\n");
        printf(" Observacion      : Condiciones bajo el punto de congelacion.\n");
    } else if (temperatura >= 0.0f && temperatura <= 20.0f) {
        printf("FRIIO (0 C a 20 C)\n");
        printf(" Observacion      : Rango termico bajo, caracteristico de zonas andinas.\n");
    } else {
        printf("TEMPLADO (> 20 C)\n");
        printf(" Observacion      : Rango termico moderado y calido.\n");
    }
    printf("----- \n");

    return 0;
}
```

Evidencia de Ejecución en Consola:

Terminal Execution - Ejercicio 9 (C)

```
=====
CLASIFICACION DE TEMPERATURAS CLIMATICAS
=====
Ingrese la temperatura registrada por el sensor (en C):
----- RESULTADO DE EVALUACION -----
Temperatura sensada: 14.50 C
Clasificacion      : FRI0 (0 C a 20 C)
Observacion       : Rango termico bajo, caracteristico de zonas andinas.
-----
```

Ejercicio 10: Ciclo for, Media Aritmética y Observaciones Superiores (C++)

Enunciado: Un investigador dispone de las mediciones de una variable para 10 observaciones. Desarrolle un programa en C++ que utilice un ciclo for para ingresar los valores, calcule la media aritmética y determine cuántas observaciones se encuentran por encima de dicha media.

Análisis y Fundamentación Algorítmica:

En C++, se utiliza el flujo de entrada `cin` y salida `cout`. Un bucle `for` recopila las 10 observaciones y acumula la sumatoria. Se calcula la media y un segundo ciclo contabiliza cuántas mediciones son estrictamente mayores al promedio.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio10.cpp>

Código Fuente (C++):

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    const int N = 10;
    double mediciones[N];
    double suma = 0.0;
    double media = 0.0;
    int conteo_superiores = 0;

    cout << "===== " << endl;
    cout << "  ANALISIS ESTADISTICO DE 10 MEDICIONES (C++)    " << endl;
    cout << "===== " << endl;

    // Ciclo for para ingresar los valores
    for (int i = 0; i < N; i++) {
        cout << "Ingrese la medicion [" << (i + 1) << "/" << N << "]: ";
        cin >> mediciones[i];
        suma += mediciones[i];
    }

    // Calculo de la media aritmetica
    media = suma / N;

    // Determinar cuantas observaciones estan por encima de la media
    for (int i = 0; i < N; i++) {
        if (mediciones[i] > media) {
            conteo_superiores++;
        }
    }

    cout << fixed << setprecision(2);
    cout << "\n----- RESUMEN ESTADISTICO -----" << endl;
    cout << " Suma total acumulada          : " << suma << endl;
    cout << " Media aritmetica del grupo      : " << media << endl;
    cout << " Observaciones mayores a la media: " << conteo_superiores << " de " << N << endl;
    cout << "\nDetalle de observaciones y clasificacion respecto a la media:" << endl;
    for (int i = 0; i < N; i++) {
        cout << " Obs " << setw(2) << (i + 1) << ": " << setw(7) << mediciones[i];
        if (mediciones[i] > media) {
            cout << " -> Por encima de la media (+)" << endl;
        }
    }
}
```

```

    } else if (mediciones[i] < media) {
        cout << "    -> Por debajo de la media (-)" << endl;
    } else {
        cout << "    -> Igual a la media" << endl;
    }
}
cout << "-----" << endl;

return 0;
}

```

Evidencia de Ejecución en Consola:

```

Terminal Execution - Ejercicio 10 (C++)
-----
ANÁLISIS ESTADÍSTICO DE 10 MEDICIONES (C++)
-----
Ingresar la medicion (1/10): Ingresar la medicion (2/10): Ingresar la medicion (3/10): Ingresar la medicion (4/10): Ingresar la medicion (5/10): Ingresar la medicion (6/10): Ingresar la medicion (7/10): Ingresar la medicion (8/10): Ingresar la medicion (9/10): Ingresar la medicion (10/10):
-----
RESUMEN ESTADÍSTICO
-----
Suma total acumulada      : 147.00
Media aritmetica del grupo : 14.70
Observaciones mayores a la media: 5 de 10

Detalle de observaciones y clasificación respecto a la media:
Obs 1: 12.50 -> Por debajo de la media (-)
Obs 2: 15.00 -> Por encima de la media (+)
Obs 3: 18.20 -> Por encima de la media (+)
Obs 4:  9.00 -> Por debajo de la media (-)
Obs 5: 14.00 -> Por debajo de la media (-)
Obs 6: 10.50 -> Por encima de la media (+)
Obs 7: 11.00 -> Por debajo de la media (-)
Obs 8: 19.50 -> Por encima de la media (+)
Obs 9: 13.50 -> Por debajo de la media (-)
Obs 10: 17.00 -> Por encima de la media (+)
-----

```

Ejercicio 11: Control de Autenticación con Ciclo while (C++)

Enunciado: Durante la carga manual de datos, el sistema requiere una clave numérica de acceso. Desarrolle un programa en C++ que solicite repetidamente la contraseña mediante un ciclo while, contabilice el número de intentos realizados y finalice cuando se ingrese la clave correcta.

Análisis y Fundamentación Algorítmica:

Se implementa un ciclo condicional indeterminado `while(!acceso_concedido)` con contador de intentos acumulativo. Al acertar la clave de acceso (2026), se habilita el sistema y se imprime el resumen de accesos.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio11.cpp>

Código Fuente (C++):

```
#include <iostream>

using namespace std;

int main() {
    const int CLAVE_CORRECTA = 2026;
    int clave_ingresada = 0;
    int intentos = 0;
    bool acceso_concedido = false;

    cout << "===== " << endl;
    cout << "  SISTEMA DE AUTENTICACION PARA CARGA DE DATOS (C++) " << endl;
    cout << "===== " << endl;
    cout << "(Clave de acceso predefinida: 2026)" << endl << endl;

    // Ciclo while para solicitar repetidamente la clave
    while (!acceso_concedido) {
        intentos++;
        cout << "Intento #" << intentos << " - Ingrese la clave numerica de acceso: ";
        cin >> clave_ingresada;

        if (clave_ingresada == CLAVE_CORRECTA) {
            acceso_concedido = true;
            cout << "\n[!] ACCESO CONCEDIDO." << endl;
            cout << "  Bienvenido al modulo de carga de datos." << endl;
        } else {
            cout << "[-] Clave incorrecta. Acceso denegado. Intente nuevamente.\n" << endl;
        }
    }

    cout << "----- " << endl;
    cout << " Resumen de inicio de sesion:" << endl;
    cout << " Numero total de intentos realizados: " << intentos << endl;
    cout << " Estado final: Autenticacion exitosa." << endl;
    cout << "===== " << endl;

    return 0;
}
```

Evidencia de Ejecución en Consola:

```
=====
SISTEMA DE AUTENTICACION PARA CARGA DE DATOS (C++)
=====
(Clave de acceso predefinida: 2026)

Intento #1 - Ingrese la clave numerica de acceso: [-] Clave incorrecta. Acceso denegado. Intente nuevamente.

Intento #2 - Ingrese la clave numerica de acceso: [-] Clave incorrecta. Acceso denegado. Intente nuevamente.

Intento #3 - Ingrese la clave numerica de acceso:
[!] ACCESO CONCEDIDO.
    Bienvenido al modulo de carga de datos.
=====
Resumen de inicio de sesion:
Numero total de intentos realizados: 3
Estado final: Autenticacion exitosa.
=====
```

Ejercicio 12: Control de Secuencias con continue y break (C++)

Enunciado: Desarrolle un programa en C++ que procese una secuencia de 10 mediciones. Los valores negativos deben considerarse inválidos y omitirse mediante continue. Si se introduce el valor 999, el procesamiento debe finalizar mediante break. Al terminar, muestre cuántos valores válidos fueron procesados.

Análisis y Fundamentación Algorítmica:

Se ilustra el uso de sentencias de salto: continue para saltar la iteración y descartar valores negativos sin computarlos, y break para interrumpir inmediatamente el ciclo ante el código de parada centinela 999.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio12.cpp>

Código Fuente (C++):

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    const int MAX_MEDICIONES = 10;
    int validos = 0;
    double valor;
    vector<double> lista_validos;

    cout << "===== " << endl;
    cout << " PROCESAMIENTO DE MEDICIONES CON CONTINUE Y BREAK " << endl;
    cout << "===== " << endl;
    cout << "Instrucciones: Se esperan hasta 10 mediciones.\n"
        << "- Valores negativos: Se omiten con continue.\n"
        << "- Valor 999 : Finaliza la captura con break.\n" << endl;

    for (int i = 1; i <= MAX_MEDICIONES; i++) {
        cout << "Entrada " << i << " de " << MAX_MEDICIONES << " - Ingrese medicion: ";
        cin >> valor;

        // Finalizacion inmediata si se introduce 999 mediante break
        if (valor == 999) {
            cout << ">> [BREAK] Se introdujo el codigo de parada 999. Finalizando proceso..." << endl;
            break;
        }

        // Omitir valores negativos considerandolos invalidos mediante continue
        if (valor < 0) {
            cout << ">> [CONTINUE] Medicion negativa (" << valor << ") invalida. Se omite." << endl;
            continue;
        }

        // Si es valido, se contabiliza y almacena
        validos++;
        lista_validos.push_back(valor);
        cout << " -> Medicion " << valor << " registrada correctamente como valida." << endl;
    }

    cout << "\n----- RESUMEN FINAL DEL PROCESAMIENTO ----" << endl;
    cout << " Total de valores validos procesados: " << validos << endl;
    cout << " Valores validos registrados: ";
    if (validos == 0) {
```

```

        cout << "Ninguno";
    } else {
        for (size_t j = 0; j < lista_validos.size(); j++) {
            cout << lista_validos[j] << (j + 1 < lista_validos.size() ? ", " : "");
        }
    }
    cout << "\n-----" << endl;

    return 0;
}

```

Evidencia de Ejecución en Consola:

```

Terminal Execution - Ejercicio 12 (C++)

=====
PROCESAMIENTO DE MEDICIONES CON CONTINUE Y BREAK
=====
Instrucciones: Se esperan hasta 10 mediciones.
- Valores negativos: Se omiten con continue.
- Valor 999      : Finaliza la captura con break.

Entrada 1 de 10 - Ingrese medicion:  -> Medicion 15.2 registrada correctamente como valida.
Entrada 2 de 10 - Ingrese medicion: >> [CONTINUE] Medicion negativa (-4) invalida. Se omite.
Entrada 3 de 10 - Ingrese medicion:  -> Medicion 22.8 registrada correctamente como valida.
Entrada 4 de 10 - Ingrese medicion:  -> Medicion 18.1 registrada correctamente como valida.
Entrada 5 de 10 - Ingrese medicion: >> [CONTINUE] Medicion negativa (-9.5) invalida. Se omite.
Entrada 6 de 10 - Ingrese medicion:  -> Medicion 30 registrada correctamente como valida.
Entrada 7 de 10 - Ingrese medicion: >> [BREAK] Se introdujo el codigo de parada 999. Finalizando proceso...

----- RESUMEN FINAL DEL PROCESAMIENTO -----
Total de valores validos procesados: 4
Valores validos registrados: 15.2, 22.8, 18.1, 30
-----

```

Ejercicio 13: Arreglos y Métricas Estadísticas de Edades

Enunciado: Un conjunto de datos contiene las edades de 10 participantes de un estudio. Desarrolle un programa en C que almacene los valores en un arreglo y determine la edad mínima, edad máxima, media y número de participantes cuya edad supera la media del grupo.

Análisis y Fundamentación Algorítmica:

Se almacena la muestra en un arreglo unidimensional `int edades[10]`. Se procesa en una sola pasada la suma y la búsqueda de extremos mínimo y máximo. Finalmente, se filtran los participantes que superan la edad media calculada.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio13.c>

Código Fuente (C):

```
#include <stdio.h>

#define N 10

int main() {
    int edades[N];
    int min_edad, max_edad;
    double suma = 0.0, media = 0.0;
    int sobre_media = 0;

    printf("=====\n");
    printf("    ESTUDIO DEMOGRAFICO: ANALISIS DE 10 EDADES      \n");
    printf("=====\n");

    for (int i = 0; i < N; i++) {
        printf("Ingrese la edad del participante [%d/%d]: ", i + 1, N);
        scanf("%d", &edades[i]);
        suma += edades[i];
    }

    // Inicializar min y max con el primer elemento
    min_edad = edades[0];
    max_edad = edades[0];

    for (int i = 1; i < N; i++) {
        if (edades[i] < min_edad) min_edad = edades[i];
        if (edades[i] > max_edad) max_edad = edades[i];
    }

    media = suma / N;

    for (int i = 0; i < N; i++) {
        if (edades[i] > media) {
            sobre_media++;
        }
    }

    printf("\n----- RESULTADOS ESTADISTICOS ----- \n");
    printf(" Lista de edades ingresadas: ");
    for (int i = 0; i < N; i++) {
        printf("%d%s", edades[i], (i < N - 1) ? ", " : "\n");
    }
    printf(" Edad minima en el grupo      : %d anios\n", min_edad);
    printf(" Edad maxima en el grupo      : %d anios\n", max_edad);
}
```

```

printf(" Edad media del grupo      : %.2f años\n", media);
printf(" Participantes con edad > media: %d participantes\n", sobre_media);
printf("-----\n");

return 0;
}

```

Evidencia de Ejecución en Consola:

```

Terminal Windows - Ejercicio 19.01
C:\Users\user> gcc ejercicio19_01.c -o ejercicio19_01.exe
C:\Users\user> ./ejercicio19_01.exe
Ingrese la edad del participante [1]:18; Ingrese la edad del participante [2]:18; Ingrese la edad del participante [3]:18; Ingrese la edad del participante [4]:18; Ingrese la edad del participante [5]:18; Ingrese la edad del participante [6]:18; Ingrese la edad del participante [7]:18; Ingrese la edad del participante [8]:18; Ingrese la edad del participante [9]:18; Ingrese la edad del participante [10]:18
Lista de edades ingresadas: 18 18 18 18 18 18 18 18 18 18
Edad promedio del grupo: 18.00 años
Edad mayor que el grupo: 0.00 años
Participantes con edad > media: 0 participantes

```

Ejercicio 14: Comparación Empírica de Complejidades $O(n)$ vs $O(n^2)$

Enunciado: Desarrolle un programa en C que permita ingresar un valor n y compare dos procedimientos: el primero debe realizar un solo recorrido de n elementos y el segundo debe utilizar dos ciclos anidados de n iteraciones. Utilice contadores para estimar el número de operaciones realizadas y relacione los resultados con complejidades aproximadas $O(n)$ y $O(n^2)$.

Análisis y Fundamentación Algorítmica:

Se evalúa experimentalmente el costo asintótico: un recorrido lineal realiza exactamente n operaciones ($O(n)$), mientras que dos bucles anidados realizan $n \times n = n^2$ operaciones ($O(n^2)$). El factor de crecimiento relativo es exactamente n , demostrando el impacto del crecimiento cuadrático.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio14.c>

Código Fuente (C):

```
#include <stdio.h>

int main() {
    int n;
    long long contador_lineal = 0;
    long long contador_cuadratico = 0;

    printf("=====\n");
    printf("    ANALISIS DE COMPLEJIDAD: O(n) vs O(n^2)      \n");
    printf("=====\n");

    printf("Ingrese la cantidad de elementos n (ej. 10, 50, 100, 500): ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Error: Ingrese un valor entero positivo para n.\n");
        return 1;
    }

    // Procedimiento 1: Un solo recorrido de n elementos -> O(n)
    for (int i = 0; i < n; i++) {
        contador_lineal++;
    }

    // Procedimiento 2: Dos ciclos anidados de n iteraciones -> O(n^2)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            contador_cuadratico++;
        }
    }

    double ratio = (contador_lineal > 0) ? ((double)contador_cuadratico / (double)contador_lineal) :

    printf("\n+-----+-----+\n");
    printf("| Parametro / Procedimiento | Valor | \n");
    printf("+-----+-----+\n");
    printf("| Tamano de entrada (n)      | %-18d | \n", n);
    printf("| Operaciones Lineales O(n)   | %-18lld | \n", contador_lineal);
    printf("| Operaciones Cuadraticas O(n^2) | %-18lld | \n", contador_cuadratico);
    printf("| Factor de escala (O(n^2)/O(n)) | %-18.2f | \n", ratio);
    printf("+-----+-----+\n");

    printf("\nInterpretacion Teorico-Practica:\n");
```

```

printf("1. En O(n), el crecimiento del tiempo es lineal: se realizaron exactamente n = %d ops.\n",
printf("2. En O(n^2), el crecimiento es cuadratico: se realizaron exactamente n^2 = %lld ops.\n",
printf("3. Notese que el factor de escala es exactamente n = %.0f, lo que demuestra que al duplicar
printf("    el procedimiento O(n^2) incrementa su costo computacional en un factor cuadratico (x4)
printf("=====\\n");

return 0;
}

```

Evidencia de Ejecución en Consola:

```

Terminal Execution - Ejercicio 14 (C)

=====
ANALISIS DE COMPLEJIDAD: O(n) vs O(n^2)
=====
Ingrese la cantidad de elementos n (ej. 10, 50, 100, 500):
+-----+
| Parametro / Procedimiento | Valor |
+-----+
| Tamano de entrada (n)     | 50    |
| Operaciones Lineales O(n)  | 50    |
| Operaciones Cuadraticas O(n^2) | 2500  |
| Factor de escala (O(n^2)/O(n)) | 50.00 |
+-----+

Interpretacion Teorico-Practica:
1. En O(n), el crecimiento del tiempo es lineal: se realizaron exactamente n = 50 ops.
2. En O(n^2), el crecimiento es cuadratico: se realizaron exactamente n^2 = 2500 ops.
3. Notese que el factor de escala es exactamente n = 50, lo que demuestra que al duplicar n,
    el procedimiento O(n^2) incrementa su costo computacional en un factor cuadratico (x4).
=====

```



Ejercicio 15: Modularidad y Funciones Estadísticas

Enunciado: Un pequeño conjunto de datos contiene tres mediciones obtenidas de una misma variable. Desarrolle un programa en C que implemente una función `calcular_promedio` para recibir las tres mediciones, calcular su media y retornar el resultado. Además, implemente una segunda función que determine cuántas de las mediciones se encuentran por encima del promedio.

Análisis y Fundamentación Algorítmica:

Se aplica la modularización en C definiendo: 1) `calcular_promedio` que computa y retorna la media aritmética; y 2) `contar_superiores_promedio` que cuenta cuántas mediciones sobrepasan el umbral de la media calculada.

Código en GitHub: <https://github.com/henrymamani-alt/henry-ejercicios/blob/main/ejercicio15.c>

Código Fuente (C):

```
#include <stdio.h>

// Funcion 1: Recibe 3 mediciones y calcula su media aritmetica
double calcular_promedio(double m1, double m2, double m3) {
    return (m1 + m2 + m3) / 3.0;
}

// Funcion 2: Determina cuantas mediciones estan por encima del promedio
int contar_superiores_promedio(double m1, double m2, double m3, double promedio) {
    int contador = 0;
    if (m1 > promedio) contador++;
    if (m2 > promedio) contador++;
    if (m3 > promedio) contador++;
    return contador;
}

int main() {
    double m1, m2, m3;

    printf("=====\n");
    printf("  MODULARIDAD: PROMEDIO Y COMPARACION DE 3 MUESTRAS \n");
    printf("=====\n");

    printf("Ingrese la medicion 1: ");
    scanf("%lf", &m1);

    printf("Ingrese la medicion 2: ");
    scanf("%lf", &m2);

    printf("Ingrese la medicion 3: ");
    scanf("%lf", &m3);

    // Invocacion de la funcion calcular_promedio
    double promedio = calcular_promedio(m1, m2, m3);

    // Invocacion de la funcion contar_superiores_promedio
    int mayores = contar_superiores_promedio(m1, m2, m3, promedio);

    printf("\n----- RESUMEN DE RESULTADOS ----- \n");
    printf(" Mediciones ingresadas : [%.2f, %.2f, %.2f]\n", m1, m2, m3);
    printf(" Promedio calculado      : %.4f\n", promedio);
    printf(" Mediciones > promedio : %d de 3\n", mayores);
}
```

```

printf("\nDetalle individual:\n");
printf(" - Medicion 1 (%.2f): %s\n", m1, (m1 > promedio) ? "SUPERIOR al promedio" : "Menor o igual");
printf(" - Medicion 2 (%.2f): %s\n", m2, (m2 > promedio) ? "SUPERIOR al promedio" : "Menor o igual");
printf(" - Medicion 3 (%.2f): %s\n", m3, (m3 > promedio) ? "SUPERIOR al promedio" : "Menor o igual");
printf("-----\n");

return 0;
}

```

Evidencia de Ejecución en Consola:

```

Terminal Execution - Ejercicio 15 (C)

=====
MODULARIDAD: PROMEDIO Y COMPARACION DE 3 MUESTRAS
=====
Ingrese la medicion 1: Ingrese la medicion 2: Ingrese la medicion 3:
----- RESUMEN DE RESULTADOS -----
Mediciones ingresadas : [14.20, 18.60, 15.40]
Promedio calculado    : 16.0667
Mediciones > promedio : 1 de 3

Detalle individual:
- Medicion 1 (14.20): Menor o igual al promedio
- Medicion 2 (18.60): SUPERIOR al promedio
- Medicion 3 (15.40): Menor o igual al promedio
-----

```

Conclusiones Generales

1. **Optimización y Tipado de Datos:** La elección meticulosa de tipos de datos primitivos en C (`int`, `float`, `double`, `char`, `bool`) garantiza la adecuada gestión de memoria y previene imprecisiones numéricas en el procesamiento cuantitativo de observaciones estadísticas.
2. **Control de Pérdida en Casting:** La conversión explícita entre datos en coma flotante y enteros debe tratarse con rigurosidad matemática, considerando siempre el residuo decimal truncado para no sesgar el análisis de sensores climáticos e industriales.
3. **Complejidad Asintótica $O(n)$ frente a $O(n^2)$:** A través de contadores de operaciones empíricas se demostró la diferencia entre recorridos lineales y bucles anidados. Se comprobó que ante un factor de escala n , el costo operativo en $O(n^2)$ se magnifica por dicho factor, justificando la búsqueda constante de estructuras de datos más eficientes.
4. **Modularidad y Mantenibilidad:** La implementación de funciones modulares en C y flujos modernos en C++ permite una separación clara de responsabilidades, facilitando la verificación, pruebas unitarias y futura escalabilidad del software.